DEVOPS ENGINEERING MASTERY

COMPUTER SYSTEMS & LINUX ENGINEERING

কম্পিউটার সিস্টেম, Operating Systems ও Linux Administration

VOLUME 01 • FIRST EDITION

```
nahid@ubuntu-server:~$ uname -a
Linux ubuntu-server 6.8.0 • x86_64 • Ubuntu 24.04 LTS
nahid@ubuntu-server:~$ systemctl status nginx
● nginx.service – active (running)
nahid@ubuntu-server:~$ sudo apt update && sudo apt upgrade -y
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

Companion Workbook • 15 Labs • 10 Assignments • 3 Projects

Practical Lab & Assignment Workbook

DEVOPS ENGINEERING MASTERY — Volume 01 (Companion Workbook). First Edition, 8 November 2026. Ubuntu Server 24.04 LTS. This workbook collects Appendix A (15 guided labs), Appendix B (10 independent assignments) and Appendix C (3 major projects) from the textbook, so you can work through them beside your virtual machine without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab command on a production server. Solutions and marking guidance are in the separate Quiz & Answer Guide — attempt each assignment before looking.

পরিশিষ্ট A — Guided Labs (গাইডেড ল্যাব)

এই অংশের ১৫টি ল্যাব বইয়ের ২০টি অধ্যায়ের হাতে-কলমে প্রয়োগ। প্রতিটি ল্যাব Ubuntu Server 24.04 LTS-এ পরীক্ষা করার জন্য তৈরি। কোনো ল্যাবেই ধ্বংসাত্মক কমান্ড অন্ধভাবে চালাতে বলা হয়নি; যেখানে ঝুঁকি আছে, সেখানে সতর্কতা ও নিরাপদ বিকল্প দেওয়া হয়েছে। সব expected output **illustrative (উদাহরণমূলক)** — আপনার সিস্টেমে নাম, সংখ্যা, তারিখ বা UUID ভিন্ন হতে পারে। Lab শুরু করার আগে VirtualBox-এ একটি snapshot নিন, যাতে ভুল হলে ফিরে আসা যায়।

> **সতর্কতা:** Lab-এ `rm`, `mkfs`, `dd`, `chmod 777`, firewall পরিবর্তন — এসব কমান্ড শুধু নিজের VM-এ চালাবেন। হোস্ট কম্পিউটার বা অন্য কারও সার্ভারে নয়। Root হিসেবে কাজ করার আগে কমান্ডটি দুইবার পড়ুন।

> **কীভাবে ল্যাব করবেন:** প্রতিটি ল্যাবের ধাপগুলো ক্রমানুসারে করুন। একটি ধাপের verification না মিললে পরের ধাপে যাবেন না — Troubleshooting অংশ দেখুন। প্রতিটি ল্যাব শেষে Cleanup অংশটি করুন, যাতে পরের ল্যাব পরিষ্কার অবস্থা থেকে শুরু হয়।

Lab 01 — VirtualBox-এ Ubuntu Server 24.04 LTS ইনস্টল ও প্রথম Boot

> **Scenario:** আপনি ঢাকার একটি বিশ্ববিদ্যালয়ের ছাত্র। আপনার ল্যাপটপে Windows আছে, কিন্তু Linux শিখতে একটি নিরাপদ আলাদা পরিবেশ দরকার। হোস্ট OS নষ্ট না করে VirtualBox-এ Ubuntu Server চালাবেন।

শেখার উদ্দেশ্য: VM তৈরি, Ubuntu Server 24.04 LTS ইনস্টল, প্রথম login, নেটওয়ার্ক ও SSH যাচাই করতে পারবেন।

Prerequisites: অধ্যায় ০৭। হোস্টে কমপক্ষে 4 GB RAM, 25 GB ফাঁকা ডিস্ক।

Tools / Environment: VirtualBox 7.x, Ubuntu Server 24.04 LTS ISO (ubuntu.com থেকে), NAT নেটওয়ার্ক।

Initial state: হোস্টে কোনো VM নেই; ISO ডাউনলোড করা আছে।

ধাপসমূহ

1. VirtualBox-এ New VM তৈরি করুন: নাম `ubuntu-lab`, RAM 2048 MB, Disk 20 GB (VDI, dynamically allocated)।

2. Settings → Storage-এ ISO যুক্ত করুন, Network NAT রাখুন, Start করুন।
3. Installer-এ ভাষা English, Keyboard layout, নেটওয়ার্ক DHCP, Storage: Use an entire disk (LVM নয়, প্রথমবারের জন্য সরল রাখুন) নির্বাচন করুন।
4. Profile: আপনার নাম, সার্ভারের নাম `ubuntu-lab`, username (যেমন `nahid`), শক্তিশালী পাসওয়ার্ড দিন। OpenSSH server ইনস্টল করতে টিক দিন।
5. Install শেষে Reboot করুন, ISO detach করুন, username/password দিয়ে login করুন।

```
lsb_release -a
uname -r
ip a
systemctl status ssh --no-pager
sudo apt update
```

কমান্ড ব্যাখ্যা: `lsb_release -a` — distribution ও version দেখায়; `uname -r` — চলমান kernel version; `ip a` — নেটওয়ার্ক interface ও IP ঠিকানা; `systemctl status ssh` — SSH service চলছে কি না; `sudo apt update` — package তালিকা হালনাগাদ করে, কিছু ইনস্টল করে না।

Expected output (illustrative):

```
Description:    Ubuntu 24.04 LTS
Release:        24.04
Linux 6.8.0-xx-generic
2: enp0s3: inet 10.0.2.15/24
Active: active (running) – OpenBSD Secure Shell server
```

Verification: `lsb_release -a`-তে 24.04 দেখা যায়; `ip a`-তে IP আছে; `ping -c 3 8.8.8.8` সফল হয়।

সাধারণ ব্যর্থতা: ISO boot না হলে Storage-এ ISO ঠিকমতো যুক্ত হয়েছে কি না দেখুন; VT-x/AMD-V BIOS-এ বন্ধ থাকলে VirtualBox ত্রুটি দেবে — BIOS থেকে virtualization চালু করুন।

> **Troubleshooting:** Login-এ ভুল password হলে Caps Lock দেখুন। নেটওয়ার্ক না পেলে VM Settings → Network → NAT ও Cable Connected টিক আছে কি না দেখুন।

Cleanup / Reset: VM বন্ধ করে VirtualBox-এ Snapshot নিন — নাম `fresh-install`। এটি পরের সব ল্যাবের নিরাপদ ফিরতি পয়েন্ট।

Lab 02 — Filesystem Exploration: কোথায় কী থাকে

> **Scenario:** নতুন সার্ভারে login করে আপনি জানতে চান — configuration কোথায়, log কোথায়, ব্যবহারকারীর ফাইল কোথায়। ভুল ডিরেক্টরিতে ফাইল তৈরি করে সিস্টেম নষ্ট করতে চান না।

শেখার উদ্দেশ্য: Filesystem Hierarchy Standard (FHS) অনুযায়ী প্রধান ডিরেক্টরিগুলো চিনতে ও `ls`, `find` দিয়ে অনুসন্ধান করতে পারবেন।

Prerequisites: অধ্যায় ০৮। Lab 01 সম্পন্ন।

Tools / Environment: Ubuntu Server 24.04 LTS VM, সাধারণ user।

Initial state: Fresh install, home ডিরেক্টরি প্রায় খালি।

ধাপসমূহ

1. Root ও home ঘুরে দেখুন।
2. প্রতিটি প্রধান ডিরেক্টরির ভেতরে কী আছে তালিকা করুন।
3. `/etc`-তে SSH ও hostname সংক্রান্ত ফাইল খুঁজুন।
4. `/var/log`-এ সাম্প্রতিক log ফাইলের আকার দেখুন।

```
pwd; ls -la /
ls -ld /etc /home /var /tmp /usr /opt /boot /dev /proc /sys
ls /etc | head -20
ls -lh /var/log/ | head -20
find /etc -maxdepth 1 -name "*ssh*" -o -maxdepth 1 -name "hostname*"
df -h / /home
```

কমান্ড ব্যাখ্যা: `ls -ld` — ডিরেক্টরি নিজের তথ্য দেখায়, ভেতরের ফাইল নয়; `ls -lh` — human-readable আকার; `find` — শর্ত মিলিয়ে ফাইল খোঁজে; `df -h` — মাউন্ট করা filesystem-এর ফাঁকা জায়গা।

Expected output (illustrative): `/etc`-তে `ssh/`, `hostname`, `passwd` দেখা যায়; `/var/log/`-তে `syslog`, `auth.log`, `apt/`; `df -h`-তে `/` প্রায় 20 GB, ব্যবহৃত অল্প।

Verification: `cat /etc/hostname`-তে `ubuntu-lab` মেলে; `cat /proc/cpuinfo | head` CPU তথ্য দেয় — বুঝুন `/proc` আসল ফাইল নয়, kernel-এর live view।

সাধারণ ব্যর্থতা: `/root`-এ ঢুকতে Permission denied — এটি স্বাভাবিক, root-এর home সাধারণ user-এর জন্য বন্ধ। `sudo ls /root` দিয়ে যাচাই করুন।

> **সতর্কতা:** `/etc`, `/boot`, `/proc`, `/sys`-এ কিছু মুছবেন বা সম্পাদনা করবেন না — এগুলো সিস্টেমের প্রাণ। অনুশীলন শুধু `~/lab02` ডিরেক্টরিতে করুন।

Cleanup / Reset: `mkdir -p ~/lab02 && cd ~/lab02`-তে তৈরি করা পরীক্ষামূলক ফাইল মুছে ফেলুন: `rm -rf ~/lab02`।

Lab 03 — Shell, Pipes ও Text Processing: Log থেকে তথ্য বের করা

> **Scenario:** একটি ওয়েব সার্ভারের access log-এ হাজার লাইন আছে। আপনাকে জানতে হবে — কোন IP সবচেয়ে বেশি request করেছে, কয়টি 404 error, কোন URL জনপ্রিয়। GUI নয়, shell দিয়েই উত্তর বের করতে হবে।

শেখার উদ্দেশ্য: pipe, redirection, `grep`, `sort`, `uniq`, `awk`, `cut` দিয়ে বাস্তব log বিশ্লেষণ করতে পারবেন।

Prerequisites: অধ্যায় ০৯–১০।

Tools / Environment: Ubuntu VM, Bash।

Initial state: `~/lab03` খালি।

ধাপসমূহ

1. একটি নমুনা log ফাইল তৈরি করুন।
2. Error লাইন গুনুন, IP অনুযায়ী সাজান, সবচেয়ে বেশি request করা IP বের করুন।
3. ফলাফল একটি ফাইলে সংরক্ষণ করুন।

```
mkdir -p ~/lab03 && cd ~/lab03
cat > access.log << 'EOF'
192.168.1.10 - - [08/Nov/2026] "GET /index.html HTTP/1.1" 200
192.168.1.20 - - [08/Nov/2026] "GET /missing.html HTTP/1.1" 404
192.168.1.10 - - [08/Nov/2026] "GET /about.html HTTP/1.1" 200
192.168.1.20 - - [08/Nov/2026] "GET /missing.html HTTP/1.1" 404
192.168.1.30 - - [08/Nov/2026] "GET /index.html HTTP/1.1" 200
EOF
wc -l access.log
grep "404" access.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr
grep -c " 200 " access.log
awk '{print $7}' access.log | sort | uniq -c | sort -nr > top-urls.txt
cat top-urls.txt
```

কমান্ড ব্যাখ্যা: `awk '{print $1}'` — প্রতি লাইনের প্রথম field (IP) নেয়; `sort | uniq -c` — গুনে সাজায়; `sort -nr` — সংখ্যা অনুযায়ী উল্টো সাজায়; `>` — output ফাইলে পাঠায়, `>>` হলে যোগ করত; heredoc `<< 'EOF'` — বহু লাইন একসাথে ফাইলে লেখে।

Expected output (illustrative): `wc -l` → 5; `grep "404"` → 2 লাইন; IP গণনায় `192.168.1.10` ও `192.168.1.20` প্রত্যেকে 2 বার; `top-urls.txt`-তে `/index.html` 2 বার সবার উপরে।

Verification: `cat top-urls.txt` ও হাতে গোনা সংখ্যা মিলিয়ে দেখুন।

সাধারণ ব্যর্থতা: `uniq`-এর আগে `sort` না করলে গণনা ভুল হবে — `uniq` শুধু পাশাপাশি একই লাইন গোনে। Quoting ভুল হলে `awk` ভিন্ন ফল দেবে; single quote ব্যবহার করুন।

> **Pro Tip:** বড় ফাইলে প্রথমে `head -20` দিয়ে গঠন দেখে নিন, তারপর পুরো ফাইলে pipe চালান। ভুল pipe-এ পুরো ফাইল নষ্ট হওয়ার ঝুঁকি কমে।

Cleanup / Reset: `cd ~ && rm -rf ~/lab03`।

Lab 04 — Users, Groups ও Permissions: একটি টিমের ফোল্ডার তৈরি

> **Scenario:** TechNova-তে দুইজন নতুন intern যোগ দিয়েছেন — `intern1` ও `intern2`। তারা `/srv/technova` শেয়ার্ড ফোল্ডারে কাজ করবেন, কিন্তু একে অপরের ব্যক্তিগত ফাইল দেখতে পারবেন না। আপনাকে user, group ও permission সাজাতে হবে।

শেখার উদ্দেশ্য: user/group তৈরি, `chmod` / `chown`, `umask` বুঝে প্রয়োগ করতে পারবেন।

Prerequisites: অধ্যায় ১১। sudo অধিকার থাকা user।

Tools / Environment: Ubuntu VM, sudo।

Initial state: `intern1`, `intern2` কেউ নেই; `/srv/technova` নেই।

ধাপসমূহ

1. Group ও user তৈরি করুন, শেয়ার্ড ডিরেক্টরি বানান।
2. মালিকানা ও permission দিন, যাচাই করুন।
3. একজন user হিসেবে লেখার পরীক্ষা করুন।

```
sudo groupadd technova
sudo useradd -m -s /bin/bash -G technova intern1
sudo useradd -m -s /bin/bash -G technova intern2
sudo passwd intern1
sudo mkdir -p /srv/technova/shared /srv/technova/private-intern1
sudo chown root:technova /srv/technova/shared
sudo chmod 2770 /srv/technova/shared
sudo chown intern1:intern1 /srv/technova/private-intern1
sudo chmod 700 /srv/technova/private-intern1
id intern1; id intern2
ls -ld /srv/technova/shared /srv/technova/private-intern1
sudo -u intern1 touch /srv/technova/shared/test-intern1.txt && echo OK
sudo -u intern2 ls /srv/technova/private-intern1 || echo "denied as expected"
```

কমান্ড ব্যাখ্যা: `useradd -m` — home ডিরেক্টরি তৈরি করে; `-G technova` — supplementary group; `chmod 2770` — setgid (2) + group rwx (7) + owner rwx (7) + other কিছুই নয় (0): নতুন ফাইল group `technova`-র হবে; `chmod 700` — শুধু মালিকের অধিকার; `sudo -u intern1` — ওই user হিসেবে কমান্ড চালায়।

Expected output (illustrative): `id intern1`-তে `groups=...technova`; `ls -ld`-তে `drwxrws--- root technova` ও `drwx------ intern1 intern1`; শেষ লাইনে `denied as expected`।

Verification: `su - intern1` দিয়ে login করে `touch /srv/technova/shared/myfile` সফল ও `touch /srv/technova/private-intern1/x` ব্যর্থ হওয়া দেখুন।

সাধারণ ব্যর্থতা: `passwd` না দিলে user login করতে পারবে না; `chmod 777` দিলে সবাই সবকিছু করতে পারবে — এটি ভুল, নিরাপত্তা ঝুঁকি। Setgid ভুলে গেলে নতুন ফাইলের group user-এর primary group হবে।

> **সতর্কতা:** কখনোই `chmod -R 777 /srv` বা `/`-তে চালাবেন না। Lab শেষে তৈরি user মুছে ফেলুন।

Cleanup / Reset:

```
sudo rm -rf /srv/technova
sudo userdel -r intern1
sudo userdel -r intern2
sudo groupdel technova
```

Lab 05 — ACLs ও sudo: Least Privilege প্রয়োগ

> **Scenario:** একজন deploy user-কে একটি নির্দিষ্ট অ্যাপ ডিরেক্টরিতে লেখার অধিকার দিতে হবে, কিন্তু তাকে পুরো group-এর সদস্য করা যাবে না। আর একজন junior admin-কে শুধু `systemctl restart nginx` চালানোর অধিকার দিতে হবে, পূর্ণ root নয়।

শেখার উদ্দেশ্য: `getfacl` / `setfacl` ও `sudoers`-এ সীমিত অধিকার দিতে পারবেন।

Prerequisites: অধ্যায় ১১–১২, Lab 04।

Tools / Environment: Ubuntu VM, sudo, `acl` package।

Initial state: `/srv/app` নেই; `deployer` ও `junior` user নেই।

ধাপসমূহ

1. ACL package ও user তৈরি করুন।
2. শুধু `deployer`-কে একটি ডিরেক্টরিতে rwx দিন, অন্যরা যেন না পায়।
3. `junior`-কে শুধু একটি কমান্ডের sudo অধিকার দিন ও পরীক্ষা করুন।

```
sudo apt update && sudo apt install -y acl
sudo useradd -m -s /bin/bash deployer
sudo useradd -m -s /bin/bash junior
sudo mkdir -p /srv/app
sudo chown root:root /srv/app
sudo chmod 750 /srv/app
sudo setfacl -m u:deployer:rwx /srv/app
sudo setfacl -d -m u:deployer:rwx /srv/app
getfacl /srv/app
sudo -u deployer touch /srv/app/deploy.txt && echo "deployer OK"
sudo -u junior touch /srv/app/no.txt || echo "junior denied as expected"
echo "junior ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx" | sudo tee /etc/sudoers.d/
junior-nginx
sudo chmod 440 /etc/sudoers.d/junior-nginx
sudo visudo -c
sudo -u junior sudo -l -U junior
```

কমান্ড ব্যাখ্যা: `setfacl -m u:deployer:rwx` — নির্দিষ্ট user-কে অধিকার দেয়, group বদলায় না; `-d` — default ACL, নতুন ফাইলেও একই অধিকার; `getfacl` — ACL দেখায়; `visudo -c` — sudoers syntax যাচাই করে, ভুল থাকলে sudo ভেঙে যেতে পারে বলে এটি বাধ্যতামূলক; `sudo -l -U junior` — ওই user কী কী sudo করতে পারে তা দেখায়।

Expected output (illustrative): `getfacl`-তে `user:deployer:rwx` ও `default:user:deployer:rwx`; `visudo -c` → `parsed OK`; `sudo -l`-তে শুধু `/usr/bin/systemctl restart nginx` দেখা যায়।

Verification: `sudo -u junior sudo systemctl restart nginx` চেষ্টা করলে (nginx না থাকলে service ত্রুটি আসবে, কিন্তু permission ত্রুটি আসবে না) — আর `sudo -u junior sudo apt update` করলে permission denied আসা উচিত।

সাধারণ ব্যর্থতা: `/etc/sudoers` সরাসরি সম্পাদনা করে syntax ভুল করলে sudo বন্ধ হয়ে যায় — সবসময় `visudo` ব্যবহার করুন। ACL কাজ না করলে filesystem-এ ACL support আছে কি না দেখুন (Ubuntu-র ext4-এ ডিফল্ট চালু থাকে)।

> **সতর্কতা:** `NOPASSWD: ALL` কখনো দেবেন না। প্রতিটি sudo অধিকার একটি নির্দিষ্ট কমান্ডে সীমিত রাখুন — এটাই least privilege।

Cleanup / Reset:

```
sudo rm /etc/sudoers.d/junior-nginx
sudo rm -rf /srv/app
sudo userdel -r deployer
sudo userdel -r junior
sudo visudo -c
```

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 06 — Process Management: ঝুলে যাওয়া প্রসেস খুঁজে থামানো

> **Scenario:** সার্ভার হঠাৎ ধীর। `top`-এ দেখা যাচ্ছে একটি প্রসেস CPU খাচ্ছে। আপনাকে দায়ী প্রসেস শনাক্ত করে নিরাপদে থামাতে হবে, পুরো সার্ভার reboot না করে।

শেখার উদ্দেশ্য: `ps`, `top` / `htop`, `kill`, signal, foreground/background job পরিচালনা করতে পারবেন।

Prerequisites: অধ্যায় ০৫, ১৩।

Tools / Environment: Ubuntu VM, `htop`।

Initial state: সিস্টেম স্বাভাবিক; কোনো test প্রসেস চলছে না।

ধাপসমূহ

1. একটি নিরীহ CPU-খরুচে test প্রসেস background-এ চালান।
2. PID খুঁজে, resource দেখে, প্রথমে নরম signal, প্রয়োজনে জোরালো signal দিয়ে থামান।
3. একটি sleep job দিয়ে `jobs`, `bg`, `fg` অনুশীলন করুন।

```
sudo apt update && sudo apt install -y htop
yes > /dev/null &
echo $!
ps aux | grep -v grep | grep yes
top -b -n 1 | head -20
kill -TERM %1
sleep 1; ps aux | grep -v grep | grep yes || echo "stopped"
sleep 300 &
jobs
bg %1
fg %1
# Ctrl+Z চেপে suspend করুন, তারপর:
# bg %1  — background-এ চালু রাখুন;  kill %1 — থামান
pstree -p | head -20
```

কমান্ড ব্যাখ্যা: `yes > /dev/null &` — `yes` অনবরত output দেয়, `&` background-এ পাঠায়; `$!` — শেষ background job-এর PID; `kill -TERM` (signal 15) — প্রসেসকে ভদ্রভাবে বন্ধ হতে বলে, ফাইল গুছিয়ে নেওয়ার সুযোগ দেয়; `kill -9` (SIGKILL) — জোর করে থামায়, শেষ উপায়; `jobs` — বর্তমান shell-এর job তালিকা; `pstree -p` — parent-child গাছ।

Expected output (illustrative): `ps aux`-তে `yes` প্রসেসের %CPU প্রায় 100; `kill -TERM`-এর পর `ps`-তে আর নেই, `stopped` ছাপা হয়।

Verification: `ps aux | grep yes` খালি; `uptime`-এ load কমে আসে।

সাধারণ ব্যর্থতা: ভুল PID-তে `kill` করলে গুরুত্বপূর্ণ service বন্ধ হতে পারে — `ps -o pid,ppid,cmd -p <PID>` দিয়ে নিশ্চিত হোন। প্রথমেই `kill -9` দেবেন না; data হারাতে পারে। Zombie প্রসেস `kill`-এ যায় না — তার parent-কে দেখতে হয়।

> **সতর্কতা:** `pkill -9` বা `killall` pattern মিলিয়ে চালাবেন না — নামের আংশিক মিলে অন্য প্রসেস মারা যেতে পারে। সবসময় PID দিয়ে, একটিকে লক্ষ্য করে kill করুন।

Cleanup / Reset: `jobs` দেখে বাকি job থাকলে `kill %1` করুন; `ps aux | grep yes` খালি নিশ্চিত করুন।

Lab 07 — APT Package Management: নিরাপদে ইনস্টল, আপডেট ও পরিষ্কার

> **Scenario:** সার্ভারে `nginx` ও `curl` দরকার। কিন্তু গত সপ্তাহে `apt upgrade` অর্ধেক অবস্থায় থেমে গিয়েছিল। আপনাকে package system সুস্থ করে নিরাপদে ইনস্টল করতে হবে।

শেখার উদ্দেশ্য: `apt` / `dpkg`, repository, dependency, ভাঙা package মেরামত করতে পারবেন।

Prerequisites: অধ্যায় ১৪।

Tools / Environment: Ubuntu VM, ইন্টারনেট।

Initial state: `nginx` ইনস্টল নেই।

ধাপসমূহ

1. Package তালিকা হালনাগাদ, package তথ্য দেখুন।
2. `nginx` ও `curl` ইনস্টল করে সংস্করণ ও ফাইল যাচাই করুন।
3. Upgrade অনুকরণ (dry-run) দেখে তারপর আসল upgrade করুন।
4. অপ্রয়োজনীয় package পরিষ্কার করুন।

```
sudo apt update
apt show nginx | head -20
apt-cache policy nginx curl
sudo apt install -y nginx curl
nginx -v
curl --version | head -1
dpkg -l | grep -E "nginx|curl"
which nginx; dpkg -L nginx | head -10
sudo apt upgrade --dry-run | head -30
sudo apt upgrade -y
sudo apt autoremove -y
sudo apt autoclean
sudo apt list --upgradable
```

কমান্ড ব্যাখ্যা: `apt update` — repository থেকে তালিকা আনে, কিছু ইনস্টল করে না; `apt-cache policy` — কোন version, কোন repository থেকে আসবে দেখায়; `dpkg -l` — ইনস্টল করা package তালিকা; `dpkg -L` — package-এর ফাইলগুলো কোথায় বসেছে; `autoremove` — আর দরকার নেই এমন dependency মুছে দেয়; `--dry-run` — না চালিয়ে কী হবে দেখায়।

Expected output (illustrative): `nginx -v` → `nginx/1.24.0 (Ubuntu)` (সংস্করণ আপনার সময় অনুযায়ী ভিন্ন হতে পারে); `curl --version`-এ সংস্করণ; `apt list --upgradable` upgrade-এর পর খালি বা প্রায় খালি।

Verification: `systemctl status nginx --no-pager` active; ব্রাউজার বা `curl http://localhost`-এ Ubuntu nginx welcome page।

সাধারণ ব্যর্থতা: `Could not get lock /var/lib/dpkg/lock` — আরেকটি apt চলছে; অপেক্ষা করুন, lock ফাইল জোর করে মুছবেন না। `dpkg was interrupted` হলে `sudo dpkg --configure -a` তারপর `sudo apt -f install`।

> **Troubleshooting:** `404 Not Found` repository ত্রুটি হলে `sudo apt update` আবার চালান; অজানা PPA যোগ করবেন না — তৃতীয় পক্ষের repository নিরাপত্তা ঝুঁকি।

Cleanup / Reset: Lab 15 পর্যন্ত nginx রেখে দিন (পরে লাগবে); শুধু এই ল্যাবের জন্য মুছতে চাইলে `sudo apt remove --purge nginx -y && sudo apt autoremove -y`।

Lab 08 — systemd: নিজের একটি Custom Service তৈরি

> **Scenario:** TechNova-র একটি ছোট Python স্ক্রিপ্ট সার্ভার চালু হলেই চলতে হবে, থেমে গেলে নিজে থেকে আবার চালু হতে হবে, আর log `journalctl`-এ দেখা যেতে হবে। `nohup` দিয়ে চালানো পেশাদার সমাধান নয় — systemd service দরকার।

শেখার উদ্দেশ্য: unit file লেখা, `systemctl enable/start/status`, `journalctl` দিয়ে service পরিচালনা করতে পারবেন।

Prerequisites: অধ্যায় ১৫, Lab 07।

Tools / Environment: Ubuntu VM, Python 3।

Initial state: `/opt/hello-app` নেই, service নেই।

ধাপসমূহ

1. একটি সহজ Python অ্যাপ ও unit file তৈরি করুন।
2. Daemon reload করে service enable ও start করুন।
3. Log দেখুন, service থামিয়ে restart আচরণ পরীক্ষা করুন।

```
sudo mkdir -p /opt/hello-app
cat > /tmp/hello.py << 'EOF'
import time
while True:
    print("hello from technova app", flush=True)
    time.sleep(10)
EOF
sudo cp /tmp/hello.py /opt/hello-app/hello.py
sudo tee /etc/systemd/system/hello-app.service > /dev/null << 'EOF'
[Unit]
```

```
Description=TechNova Hello App
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/hello-app/hello.py
Restart=on-failure
RestartSec=5
User=nobody
Group=nogroup

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable hello-app.service
sudo systemctl start hello-app.service
sudo systemctl status hello-app.service --no-pager
journalctl -u hello-app.service --no-pager -n 20
sudo systemctl restart hello-app.service
journalctl -u hello-app.service --no-pager -n 10
```

কমান্ড ব্যাখ্যা: `[Unit]` — বিবরণ ও dependency; `After=network.target` — নেটওয়ার্কের পরে শুরু; `[Service]` `ExecStart` — চালানোর কমান্ড; `Restart=on-failure` — crash হলে আবার চালু; `User=nobody` — root নয়, সীমিত user-এ চালানো (least privilege); `daemon-reload` — unit file বদলের পর systemd-কে আবার পড়তে বলে; `enable` — boot-এ চালু হবে, `start` — এখনই চালু।

Expected output (illustrative): `status`-তে `Active: active (running)`; `journalctl`-তে বারবার `hello from technova app` লাইন।

Verification: `systemctl is-enabled hello-app` → `enabled`; `ps aux | grep hello.py`-তে প্রসেস দেখা যায়।

সাধারণ ব্যর্থতা: `daemon-reload` না করলে পুরোনো unit চলবে; `ExecStart`-এ ভুল path হলে `status`-তে `failed (Result: exit-code)` — `journalctl -u hello-app -n 50` দিয়ে কারণ দেখুন। Root-এ অ্যাপ চালাবেন না — `User=` দিন।

> **সতর্কতা:** `Restart=always` ভুল configuration-এ বারবার crash-loop তৈরি করতে পারে — প্রথমে `on-failure` ব্যবহার করুন, log দেখুন।

Cleanup / Reset:

```
sudo systemctl stop hello-app.service
sudo systemctl disable hello-app.service
sudo rm /etc/systemd/system/hello-app.service /opt/hello-app/hello.py
sudo systemctl daemon-reload
sudo rm -rf /opt/hello-app /tmp/hello.py
```

Lab 09 — Disk Management: ডিস্ক ভরা, Partition ও fstab

> **Scenario:** সার্ভারের `/` 92% ভরা। আপনাকে খুঁজে বের করতে হবে কোন ডিরেক্টরি বড়, একটি অতিরিক্ত ভার্চুয়াল ডিস্ক যুক্ত করে mount করতে হবে, যাতে reboot-এর পরও থাকে।

শেখার উদ্দেশ্য: `lsblk`, `df`, `du`, mount, `/etc/fstab` ব্যবহার করতে পারবেন।

Prerequisites: অধ্যায় ১৬। Snapshot নেওয়া থাকা বাধ্যতামূলক।

Tools / Environment: Ubuntu VM; VirtualBox-এ একটি অতিরিক্ত 5 GB VDI (VM বন্ধ করে যুক্ত করুন)।

Initial state: একটি ডিস্ক (`/dev/sda`); অতিরিক্ত ডিস্ক যুক্ত হয়েছে, কিন্তু format/mount হয়নি।

ধাপসমূহ

1. ডিস্ক ও ব্যবহার দেখুন, বড় ডিরেক্টরি খুঁজুন।
2. নতুন ডিস্কে filesystem তৈরি করে `/mnt/data`-তে mount করুন।
3. `fstab`-এ UUID দিয়ে স্থায়ী করুন, reboot ছাড়া যাচাই করুন।

```
lsblk -f
df -h
sudo du -sh /var/* 2>/dev/null | sort -rh | head -10
sudo du -sh /home/* 2>/dev/null | sort -rh | head -10
# নতুন ডিস্কটি চিনুন (সাধারণত /dev/sdb, আকার দিয়ে নিশ্চিত হোন)
lsblk
sudo mkfs.ext4 /dev/sdb
sudo mkdir -p /mnt/data
sudo mount /dev/sdb /mnt/data
df -h /mnt/data
sudo blkid /dev/sdb
# blkid থেকে UUID কপি করে নিচের লাইনে বসান (উদাহরণ UUID বদলে নিন)
echo "UUID=PASTE-UUID-HERE  /mnt/data  ext4  defaults  0  2" | sudo tee -a /etc/fstab
sudo umount /mnt/data
sudo mount -a
df -h /mnt/data
ls -ld /mnt/data
```

কমান্ড ব্যাখ্যা: `lsblk -f` — block device, filesystem ও mountpoint; `du -sh` — ডিরেক্টরির মোট আকার; `mkfs.ext4` — নতুন filesystem তৈরি (আগের সব তথ্য মুছে যায়); `blkid` — UUID দেখায়; `fstab`-এ UUID ব্যবহার করলে device নাম (`sdb`) বদলে গেলেও mount ঠিক থাকে; `mount -a` — `fstab`-এর সব entry আবার mount করে, reboot ছাড়াই পরীক্ষা করা যায়।

Expected output (illustrative): `df -h /mnt/data`-তে 5 GB ডিস্ক, ব্যবহৃত প্রায় 0; `mount -a`-এর পর কোনো ত্রুটি নেই।

Verification: `findmnt /mnt/data` ও `cat /etc/fstab | grep data` মিলিয়ে দেখুন; VM reboot করে `df -h`-তে `/mnt/data` আবার আছে কি না দেখুন।

সাধারণ ব্যর্থতা: ভুল ডিস্কে `mkfs` চালালে মূল ডিস্ক মুছে যাবে — আকার ও `lsblk` দিয়ে তিনবার নিশ্চিত হোন। `fstab`-এ ভুল UUID থাকলে boot আটকে যেতে পারে — `mount -a` দিয়ে আগেই যাচাই করুন, snapshot রাখুন।

> **সতর্কতা:** `mkfs` ও `dd` ধ্বংসাত্মক। শুধু নতুন, খালি ভার্চুয়াল ডিস্কে চালান। `/dev/sda`-তে কখনো নয়।

Cleanup / Reset: `sudo umount /mnt/data`, `fstab` থেকে লাইনটি মুছে `sudo mount -a` করুন; চাইলে VM বন্ধ করে অতিরিক্ত VDI সরিয়ে দিন।

Lab 10 — Resource Monitoring: CPU, RAM, I/O Bottleneck খুঁজে বের করা

> **Scenario:** ব্যবহারকারীরা বলছেন অ্যাপ ধীর। CPU, RAM, নাকি ডিস্ক — কোনটি bottleneck, তা না জেনে RAM বাড়ানো অর্থের অপচয়। আপনাকে তথ্য দিয়ে প্রমাণ করতে হবে।

শেখার উদ্দেশ্য: `uptime`, `free`, `vmstat`, `top` / `htop`, `iostat` দিয়ে bottleneck শনাক্ত করতে পারবেন।

Prerequisites: অধ্যায় ০৫, ১৭, Lab 06।

Tools / Environment: Ubuntu VM, `sysstat` package।

Initial state: সিস্টেম স্বাভাবিক।

ধাপসমূহ

1. Baseline মাপুন (স্বাভাবিক অবস্থা)।
2. ইচ্ছাকৃত load তৈরি করে আবার মাপুন ও তুলনা করুন।
3. কোন resource চাপে আছে, তা লিখে সিদ্ধান্ত নিন।

```
sudo apt update && sudo apt install -y sysstat
uptime
free -h
vmstat 1 5
top -b -n 1 | head -20
iostat -x 1 3
# Load তৈরি (VM-এ, অল্প সময়ের জন্য)
stress --version 2>/dev/null || sudo apt install -y stress
timeout 20 stress --cpu 2 --vm 1 --vm-bytes 256M &
uptime; free -h; vmstat 1 5; iostat -x 1 3
wait
uptime; free -h
```

কমান্ড ব্যাখ্যা: `uptime`-এর load average (1, 5, 15 মিনিট) — CPU core সংখ্যার সাথে তুলনা করুন; `free -h`-তে `available` কলামই আসল ফাঁকা RAM, `free` নয়; `vmstat`-এ `si/so` (swap in/out) বেশি হলে RAM চাপে, `wa` বেশি হলে I/O অপেক্ষা; `iostat -x`-এ `%util` 90%+ হলে ডিস্ক bottleneck; `stress` — পরীক্ষার জন্য কৃত্রিম load।

Expected output (illustrative): Baseline-এ load 0.10-এর নিচে, `available` RAM পর্যাপ্ত, `%util` কম। Load চলাকালে load average 2-এর উপরে, `free`-এ available কমে, `vmstat`-এ CPU `us` বাড়ে। Load শেষে আবার স্বাভাবিক।

Verification: Baseline ও load অবস্থার সংখ্যা পাশাপাশি লিখুন — কোন কলাম বদলেছে, তা দিয়ে bottleneck-এর নাম বলুন।

সাধারণ ব্যর্থতা: শুধু `free` দেখে RAM শেষ ভাবা — Linux cache-এর জন্য RAM ব্যবহার করে, `available` দেখুন। Load average-কে শতাংশ ভাবা — এটি core সংখ্যার সাপেক্ষে বুঝতে হয় (2 core-এ load 2.0 মানে সম্পূর্ণ ব্যস্ত)।

> **Pro Tip:** একবার মেপে সিদ্ধান্ত নেবেন না। `vmstat 1 5` বা `iostat` কয়েকবার চালিয়ে ধারা (trend) দেখুন — এক মুহূর্তের spike বিভ্রান্ত করে।

Cleanup / Reset: `stress` শেষ হলে `ps aux | grep stress` খালি নিশ্চিত করুন; প্রয়োজনে `pkill stress` নয়, PID দিয়ে `kill` করুন।

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 11 — Log Troubleshooting: কেন Service থামল, Log পড়ে খুঁজুন

> **Scenario:** সকালে nginx চালু হচ্ছে না। কেউ বলছে "সার্ভার ডাউন", কিন্তু কারণ কেউ জানে না। আপনাকে অনুমান নয়, log পড়ে কারণ প্রমাণ করতে হবে।

শেখার উদ্দেশ্য: `journalctl`, `/var/log`, `dmesg` দিয়ে service failure-এর মূল কারণ (root cause) খুঁজতে পারবেন।

Prerequisites: অধ্যায় ১৫, ১৮, Lab 07–০৮।

Tools / Environment: Ubuntu VM, nginx ইনস্টল করা।

Initial state: nginx চলছে।

ধাপসমূহ

1. ইচ্ছাকৃতভাবে একটি ভুল configuration তৈরি করে nginx ভাঙুন (VM-এ, নিরাপদে)।
2. Service status, journal ও error log পড়ে কারণ শনাক্ত করুন।
3. ভুল সারিয়ে service আবার চালু করুন।

```
sudo systemctl status nginx --no-pager
sudo cp /etc/nginx/nginx.conf /tmp/nginx.conf.bak
echo "this is not valid nginx config" | sudo tee -a /etc/nginx/nginx.conf
sudo systemctl restart nginx || echo "failed as expected"
sudo systemctl status nginx --no-pager
journalctl -u nginx --no-pager -n 50
sudo tail -50 /var/log/nginx/error.log
sudo nginx -t
sudo cp /tmp/nginx.conf.bak /etc/nginx/nginx.conf
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl status nginx --no-pager
sudo tail -20 /var/log/nginx/error.log
dmesg | tail -20
```

কমান্ড ব্যাখ্যা: `nginx -t` — configuration পরীক্ষা করে, service চালু না করে; `journalctl -u nginx` — শুধু ওই unit-এর log; `tail -50` — ফাইলের শেষ ৫০ লাইন, যেখানে সাম্প্রতিক ত্রুটি থাকে; `dmesg` — kernel ring buffer, hardware/driver ত্রুটি দেখায়। প্রথমে backup নেওয়া হয়েছে বলে ভাঙা অবস্থা থেকে ফেরা যায়।

Expected output (illustrative): ভাঙা অবস্থায় `nginx -t` → `emerg` / syntax error ও লাইন নম্বর; `status` → `failed`; মেরামতের পর `nginx -t` → `syntax is ok`, `test is successful`; `status` → `active (running)`।

Verification: `curl -I http://localhost`-তে `HTTP/1.1 200 OK`; `journalctl -u nginx --since "5 minutes ago"`-তে নতুন কোনো error নেই।

সাধারণ ব্যর্থতা: শুধু `restart` বারবার চালানো — কারণ না জেনে restart করলে একই ত্রুটি ফিরে আসে। Log না পড়ে configuration অনুমান করে বদলানো। `/var/log`-এর ফাইল সরাসরি সম্পাদনা করা — log পড়ার জিনিস, বদলানোর নয়।

> **Troubleshooting:** ত্রুটির লাইনে ফাইল নাম ও লাইন নম্বর দেওয়া থাকে — সেখান থেকেই শুরু করুন। সময় (timestamp) মিলিয়ে দেখুন: failure-এর ঠিক আগের লাইনগুলোই সবচেয়ে গুরুত্বপূর্ণ।

Cleanup / Reset: `rm /tmp/nginx.conf.bak`; `sudo systemctl status nginx` active নিশ্চিত করুন।

Lab 12 — UFW Firewall ও SSH Hardening: সার্ভারের দরজা বন্ধ করা

> **Scenario:** TechNova-র সার্ভার ইন্টারনেটে যাবে। এখন সব port খোলা। আপনাকে শুধু SSH (22) ও HTTP (80) খুলে বাকি সব বন্ধ করতে হবে, আর password দিয়ে SSH login বন্ধ করে key-based login চালু করতে হবে।

শেখার উদ্দেশ্য: UFW দিয়ে firewall নিয়ম, SSH key তৈরি, `sshd_config` নিরাপদ করা ও `ss` দিয়ে listening port যাচাই করতে পারবেন।

Prerequisites: অধ্যায় ১৯, Lab 01, Lab 07।

Tools / Environment: Ubuntu VM, `ufw`, `openssh-server`।

Initial state: UFW inactive; SSH password login চালু।

ধাপসমূহ

1. বর্তমান listening port দেখুন।
2. SSH অনুমতি দিয়ে তারপর UFW চালু করুন (ক্রমটি গুরুত্বপূর্ণ)।
3. SSH key জোড়া তৈরি করে key login পরীক্ষা করুন, তারপর password login বন্ধ করুন।

```
sudo ss -tulpn
sudo ufw status verbose
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw enable
sudo ufw status verbose
sudo ss -tulpn
ssh-keygen -t ed25519 -C "nahid@ubuntu-lab" -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ls -l ~/.ssh/
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
sudo grep -E "PasswordAuthentication|PermitRootLogin" /etc/ssh/sshd_config
```

কমান্ড ব্যাখ্যা: `ss -tulpn` — listening TCP/UDP port, প্রসেস ও PID; `ufw allow OpenSSH` — SSH profile অনুযায়ী port 22 খোলে; `ufw enable`-এর আগে SSH allow না করলে নিজেই lockout হয়ে যাবেন — এই Lab-এ VM

console আছে বলে ঝুঁকি কম, কিন্তু আসল সার্ভারে এটি সবচেয়ে সাধারণ ভুল; `ssh-keygen -t ed25519` — আধুনিক, ছোট ও নিরাপদ key; `authorized_keys`-এ public key থাকলে ওই private key দিয়ে password ছাড়া login হয়; private key (`id_ed25519`) কখনো কাউকে দেবেন না, শেয়ার করবেন না।

Expected output (illustrative): `ufw status`-তে `Status: active`, `22/tcp ALLOW`, `80/tcp ALLOW`; `ss`-তে `0.0.0.0:22` ও `0.0.0.0:80` listening; `ls -l ~/.ssh/`-তে `authorized_keys` 600।

Verification: নতুন terminal থেকে `ssh -i ~/.ssh/id_ed25519 nahid@localhost` (VM-এ) password ছাড়া ঢোকে কি না দেখুন। `sudo ufw status numbered` দিয়ে নিয়মের ক্রম দেখুন।

সাধারণ ব্যর্থতা: `~/.ssh` 700 ও `authorized_keys` 600 না হলে SSH key উপেক্ষা করে — permission-ই কারণ। `sshd_config` বদলে `systemctl reload ssh` না করলে পরিবর্তন কার্যকর হয় না।

> **সতর্কতা:** এই ল্যাবে `PasswordAuthentication no` সরাসরি চালু করবেন না — প্রথমে key login পরীক্ষা করে নিশ্চিত হোন। Key হারালে ও password বন্ধ থাকলে সার্ভারে আর ঢোকা যাবে না। আসল সার্ভারে বদলের আগে `sshd_config.bak` রাখুন ও console access নিশ্চিত করুন। Private key কখনো Git, চ্যাট বা স্ক্রিনশটে দেবেন না।

Cleanup / Reset: Lab শেষে VM-এ UFW চাইলে চালু রাখুন; পরীক্ষার key মুছতে চাইলে `rm ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub` ও `authorized_keys` থেকে লাইনটি সরান। `sshd_config.bak` থেকে ফিরতে `sudo cp /etc/ssh/sshd_config.bak /etc/ssh/sshd_config && sudo systemctl reload ssh`।

Lab 13 — Backup ও Restore: tar ও rsync দিয়ে তথ্য ফিরিয়ে আনা

> **Scenario:** TechNova-র `/srv/technova`-তে গুরুত্বপূর্ণ project ফাইল আছে। ডিস্ক নষ্ট হলে বা ভুলে মুছে গেলে তথ্য ফিরিয়ে আনতে হবে। Backup নেওয়া হয়নি — এমন কথা শোনার আগেই backup ও restore পরীক্ষা করতে হবে।

শেখার উদ্দেশ্য: `tar` দিয়ে archive, `rsync` দিয়ে incremental backup, checksum যাচাই ও restore করতে পারবেন।

Prerequisites: অধ্যায় ২০, Lab 04।

Tools / Environment: Ubuntu VM, `rsync`।

Initial state: `/srv/technova`-তে কিছু নমুনা ফাইল আছে (না থাকলে তৈরি করুন)।

ধাপসমূহ

1. নমুনা তথ্য তৈরি করে tar backup নিন ও checksum লিখে রাখুন।
2. rsync দিয়ে দ্বিতীয় backup নিন, একটি ফাইল বদলে আবার rsync চালিয়ে incremental আচরণ দেখুন।
3. মূল ফাইল মুছে (VM-এ, নমুনা তথ্য) backup থেকে restore করুন।

```
sudo apt update && sudo apt install -y rsync
sudo mkdir -p /srv/technova/docs
echo "Project plan v1" | sudo tee /srv/technova/docs/plan.txt
echo "Client notes" | sudo tee /srv/technova/docs/notes.txt
sudo mkdir -p /backup
sudo tar -czvf /backup/technova-2026-11-08.tar.gz -C /srv technova
ls -lh /backup/
```

```
sha256sum /backup/technova-2026-11-08.tar.gz | sudo tee /backup/technova-2026-11-08.sha256
tar -tzvf /backup/technova-2026-11-08.tar.gz | head -20
sudo rsync -av /srv/technova/ /backup/technova-rsync/
echo "Project plan v2 - updated" | sudo tee /srv/technova/docs/plan.txt
sudo rsync -av /srv/technova/ /backup/technova-rsync/
# Restore পরীক্ষা (নমুনা তথ্য, VM-এ)
sudo rm -rf /srv/technova/docs
sudo mkdir -p /tmp/restore-test
sudo tar -xzvf /backup/technova-2026-11-08.tar.gz -C /tmp/restore-test
ls -R /tmp/restore-test
cat /tmp/restore-test/technova/docs/plan.txt
sha256sum -c /backup/technova-2026-11-08.sha256
```

কমান্ড ব্যাখ্যা: `tar -czvf` — c=create, z=gzip compress, v=verbose, f=file; `-C /srv technova` — `/srv`-তে গিয়ে `technova` archive করে, absolute path এড়ায়; `sha256sum` — backup অক্ষত আছে কি না যাচাইয়ের fingerprint; `rsync -av` — a=archive (permission, সময় ধরে রাখে), v=verbose; দ্বিতীয়বার rsync শুধু বদলে যাওয়া ফাইল কপি করে — এটাই incremental-এর মূল ধারণা; `tar -tzvf` — না খুলে ভেতরে কী আছে দেখা।

Expected output (illustrative): tar ফাইল কয়েক KB; `tar -tzvf`-তে `technova/docs/plan.txt` দেখা যায়; দ্বিতীয় rsync-এ শুধু `plan.txt` কপি হয়; restore-এর পর `plan.txt`-তে `Project plan v1` ফিরে আসে; `sha256sum -c` → `OK`।

Verification: Restore করা ফাইলের বিষয়বস্তু মূলের সাথে মিলিয়ে দেখুন; `diff -r /tmp/restore-test/technova /backup/technova-rsync` (v2 পার্থক্য থাকবে — কেন, তা বুঝুন: tar backup v1 সময়ের, rsync v2)।

সাধারণ ব্যর্থতা: Backup নেওয়া কিন্তু restore পরীক্ষা না করা — সবচেয়ে বড় ভুল; backup ফাইল ও মূল তথ্য একই ডিস্কে রাখা — ডিস্ক নষ্ট হলে দুটোই যায়; `rsync`-এ trailing slash (`/srv/technova/` বনাম `/srv/technova`) ভুল হলে গন্তব্যে অতিরিক্ত ডিরেক্টরি তৈরি হয়।

> **সতর্কতা:** `rm -rf` শুধু এই ল্যাবের নমুনা ডিরেক্টরিতে চালান। আসল তথ্যে restore পরীক্ষা করার আগে backup-এর checksum যাচাই করুন।

Cleanup / Reset: `sudo rm -rf /tmp/restore-test /backup/technova-*`; চাইলে `/srv/technova` রেখে দিন Project B-র জন্য।

Lab 14 — Disk Full Investigation: হঠাৎ ডিস্ক ভরে গেলে কী করবেন

> **Scenario:** সকালে অ্যাপ লিখতে পারছে না — `No space left on device`। সার্ভার reboot নয়, প্রথমে কোথায় জায়গা শেষ হয়েছে তা খুঁজতে হবে, তারপর নিরাপদে জায়গা খালি করতে হবে।

শেখার উদ্দেশ্য: Disk full-এর কারণ শনাক্ত, নিরাপদ cleanup, log rotation ধারণা প্রয়োগ করতে পারবেন।

Prerequisites: অধ্যায় ১৬, ১৮, Lab 09।

Tools / Environment: Ubuntu VM।

Initial state: ডিস্ক স্বাভাবিক; পরীক্ষার জন্য একটি বড় নমুনা ফাইল তৈরি করা হবে।

ধাপসমূহ

1. একটি বড় নমুনা ফাইল তৈরি করে ডিস্ক চাপ অনুকরণ করুন (ছোট আকারে, VM-এ)।
2. `df` ও `du` দিয়ে দায়ী ডিরেক্টরি খুঁজুন।
3. নিরাপদে মুছে জায়গা ফিরিয়ে আনুন; কী মুছবেন না, তা লিখুন।

```
df -h /
df -i /
sudo du -sh /* 2>/dev/null | sort -rh | head -10
sudo du -sh /var/log/* 2>/dev/null | sort -rh | head -10
sudo du -sh /tmp/* ~/.* 2>/dev/null | sort -rh | head -10
# অনুকরণ: /tmp-এ 500 MB নমুনা ফাইল (আসল ডিস্ক ভরবেন না)
fallocate -l 500M /tmp/big-test-file
df -h /
ls -lh /tmp/big-test-file
# সবচেয়ে বড় ফাইলগুলো খুঁজুন
sudo find /tmp /var/log -type f -size +10M -exec ls -lh {} \; 2>/dev/null | head -20
sudo journalctl --disk-usage
sudo journalctl --vacuum-size=100M
rm /tmp/big-test-file
df -h /
sudo apt autoremove -y; sudo apt autoclean
```

কমান্ড ব্যাখ্যা: `df -i` — inode শেষ হয়েছে কি না দেখায় (জায়গা থাকলেও inode শেষ হলে ফাইল তৈরি হয় না); `du -sh /*` — root-এর প্রতিটি ডিরেক্টরির আকার; `fallocate` — দ্রুত বড় ফাইল তৈরি করে, পরীক্ষার জন্য; `journalctl --vacuum-size=100M` — journal log 100 MB-এ নামিয়ে আনে; `find -size +10M` — বড় ফাইল খুঁজে বের করে।

Expected output (illustrative): `du`-তে `/tmp/big-test-file` 500M দেখা যায়; মুছে ফেলার পর `df -h`-তে Available আবার বাড়ে; `journalctl --disk-usage`-তে journal আকার দেখা যায়।

Verification: মুছে ফেলার আগে ও পরে `df -h /` লিখে তুলনা করুন; `touch /tmp/test-write && rm /tmp/test-write` দিয়ে লেখা যায় কি না পরীক্ষা করুন।

সাধারণ ব্যর্থতা: চলমান প্রসেসের খোলা log ফাইল মুছলে জায়গা ফেরে না — প্রসেসটি ফাইল handle ধরে রাখে; তখন service restart বা `truncate` দরকার (এই ল্যাবে `journalctl --vacuum` নিরাপদ পথ)। `/var/log`-এর সব ফাইল মুছে ফেলা — তদন্তের প্রমাণ নষ্ট হয়; logrotate-এর নিয়ম বুঝে cleanup করুন। Inode শেষ হলে ছোট ছোট অসংখ্য ফাইল (যেমন cache/session) দায়ী হতে পারে — `find` দিয়ে সংখ্যা গুনুন।

> **সতর্কতা:** `rm -rf /var/log/*` বা `/tmp`-এর সবকিছু অন্ধভাবে মুছবেন না। কী মুছছেন, কেন মুছছেন — আগে `ls` দিয়ে দেখুন।

Cleanup / Reset: `rm -f /tmp/big-test-file /tmp/test-write`; `df -h /` স্বাভাবিক নিশ্চিত করুন।

Lab 15 — Final Server Health Check: একটি সার্ভারের সম্পূর্ণ স্বাস্থ্য পরীক্ষা

> **Scenario:** আপনি TechNova-র নতুন junior sysadmin। সোমবার সকালে সাপ্তাহিক health check রিপোর্ট দিতে হবে: সার্ভার চালু আছে কি না, ডিস্ক/RAM/CPU ঠিক আছে কি না, service চলছে কি না, log-এ ত্রুটি আছে কি না, backup হয়েছে কি না, update বাকি আছে কি না।

শেখার উদ্দেশ্য: আগের সব ল্যাবের দক্ষতা একত্র করে একটি পুনরাবৃত্তিযোগ্য health-check রুটিন তৈরি ও রিপোর্ট লিখতে পারবেন।

Prerequisites: Lab 01–১৪ সবকটি (অন্তত Lab 01, 07, 08, 10, 11, 13)।

Tools / Environment: Ubuntu VM, সব আগের service।

Initial state: সার্ভার চলছে; nginx ও hello-app (যদি থাকে) চলছে।

ধাপসমূহ

1. একটি health-check স্ক্রিপ্ট তৈরি করুন।
2. চালিয়ে রিপোর্ট ফাইলে সংরক্ষণ করুন।
3. প্রতিটি অংশের ফল নিজে ব্যাখ্যা করে একটি সংক্ষিপ্ত সিদ্ধান্ত লিখুন: OK / Watch / Action needed।

```
cat > ~/health-check.sh << 'EOF'
#!/bin/bash
echo "=== TechNova Server Health Check: $(date) ==="
echo "--- Uptime & Load ---"
uptime
echo "--- Disk ---"
df -h /
df -i /
echo "--- Memory ---"
free -h
echo "--- Services ---"
systemctl is-active nginx ssh
systemctl --failed --no-pager
echo "--- Updates ---"
apt list --upgradable 2>/dev/null | head -10
echo "--- Recent Errors ---"
journalctl -p err --since "24 hours ago" --no-pager -n 20
echo "--- Listening Ports ---"
sudo ss -tulpn
echo "--- Backup ---"
ls -lh /backup/ 2>/dev/null || echo "No /backup directory found"
EOF
chmod +x ~/health-check.sh
~/health-check.sh | tee ~/health-report-$(date +%F).txt
cat ~/health-report-*.txt | head -60
```

কমান্ড ব্যাখ্যা: `systemctl --failed` — ব্যর্থ service তালিকা করে; `journalctl -p err --since "24 hours ago"` — গত ২৪ ঘণ্টার error-level log; `tee` — output একই সাথে স্ক্রিনে ও ফাইলে পাঠায়; `$(date +%F)` — ফাইলের নামে আজকের তারিখ (যেমন 2026-11-08) বসায়; স্ক্রিপ্টের প্রথম লাইন `#!/bin/bash` (shebang) বলে কোন interpreter দিয়ে চালাতে হবে।

Expected output (illustrative): প্রতিটি অংশে শিরোনামসহ ফলাফল; `systemctl is-active`-তে `active`; `--failed` খালি (বা শুধু জানা ব্যর্থ service); error log-এ নতুন গুরুতর ত্রুটি নেই; `/backup/`-তে Lab 13-এর ফাইল দেখা যায়।

Verification: রিপোর্ট ফাইলটি খুলে প্রতিটি অংশে হাতে লিখুন: OK / Watch / Action needed। অন্তত একটি Action খুঁজে বের করুন (যেমন বাকি update, বা journal আকার) ও তা করুন।

সাধারণ ব্যর্থতা: স্ক্রিপ্টে `chmod +x` না করলে Permission denied; `sudo` দরকার এমন অংশ (যেমন `ss -p`) স্ক্রিপ্টে sudo ছাড়া চালালে তথ্য অসম্পূর্ণ থাকবে — কোন অংশে sudo দরকার, তা নোট করুন। রিপোর্ট তৈরি করে কাউকে না পাঠানো — health check-এর উদ্দেশ্য সিদ্ধান্ত, শুধু ফাইল নয়।

> **Pro Tip:** এই স্ক্রিপ্টকে সাপ্তাহিক cron job-এ পরিণত করা যায়, কিন্তু প্রথমে হাতে চালিয়ে প্রতিটি লাইনের অর্থ বুঝুন। Automation বোঝার বিকল্প নয়।

Cleanup / Reset: রিপোর্ট ফাইল `~/health-report-*.txt` সংরক্ষণ করুন (Project B-র deliverable হিসেবে লাগবে); স্ক্রিপ্টটি `~/health-check.sh`-এ রেখে দিন।

পরিশিষ্ট B — Independent Assignments (স্বাধীন অ্যাসাইনমেন্ট)

নিচের ১০টি অ্যাসাইনমেন্টের সমাধান এখানে দেওয়া হয়নি — সমাধান আলাদা Answer Guide-এ আছে। প্রথমে নিজে চেষ্টা করুন, আটকে গেলে অধ্যায় ও ল্যাব দেখুন, তারপর Answer Guide মিলিয়ে নিন। প্রতিটি অ্যাসাইনমেন্ট VM-এ করুন, নিজের কাজের ধাপ ও ফলাফল একটি নোট ফাইলে লিখে রাখুন।

> **নিয়ম:** Beginner (১–৪) একা করা যাবে। Intermediate (৫–৭) করার আগে সংশ্লিষ্ট ল্যাব শেষ করুন। Advanced (৮–১০) সময় নিয়ে, log পড়ে, ধাপে ধাপে করুন — সরাসরি সমাধান খুঁজবেন না।

Beginner — অ্যাসাইনমেন্ট ১–৪

Assignment 01 — আমার সার্ভারের পরিচয়পত্র

নিজের VM-এ একটি `system-info.txt` ফাইল তৈরি করুন, যেখানে থাকবে: Ubuntu version, kernel version, CPU মডেল ও core সংখ্যা, মোট RAM, ডিস্ক আকার ও ফাঁকা জায়গা, hostname, IP ঠিকানা, uptime। প্রতিটি তথ্যের পাশে কোন কমান্ড দিয়ে পেয়েছেন, তা লিখুন। ফাইলটি `~/assignments/` ডিরেক্টরিতে রাখুন।

Assignment 02 — Log থেকে গল্প বের করা

Lab 03-এর মতো নিজে একটি ২০ লাইনের নমুনা access log তৈরি করুন (অন্তত ৩টি IP, ৩টি URL, 200/404/500 status মিশিয়ে)। তারপর shell কমান্ড দিয়ে বের করুন: (ক) মোট request, (খ) প্রতিটি status code কতবার, (গ) সবচেয়ে বেশি request করা IP, (ঘ) সবচেয়ে জনপ্রিয় URL। ফলাফল `log-report.txt`-এ লিখুন ও প্রতিটি কমান্ডের ব্যাখ্যা এক লাইনে দিন।

Assignment 03 — নিরাপদ ফাইল অপারেশন

`~/practice` ডিরেক্টরিতে ১০টি ফাইল তৈরি করুন (নামে তারিখ ও ধরন মিশিয়ে, যেমন `report-2026-11-08.txt`)। তারপর: (ক) `.txt` ফাইলগুলো `~/practice/text/`-এ সরান, (খ) একটি ফাইলের hard link ও একটি symbolic link তৈরি করে পার্থক্য পরীক্ষা করুন (মূল ফাইল মুছে কী হয়?), (গ) `tar` দিয়ে পুরো `practice` ডিরেক্টরির backup নিন, (ঘ) ভুল করে `rm` চালানোর ঝুঁকি এড়াতে আপনি কী সতর্কতা নেবেন, তা তিন লাইনে লিখুন।

Assignment 04 — User ও Permission পরিকল্পনা

একটি ছোট দলের জন্য পরিকল্পনা লিখুন (কাগজে বা Markdown ফাইলে): ২ জন developer, ১ জন designer, ১ জন intern। কে কোন group-এ থাকবে, শেয়ার্ড ফোল্ডার `/srv/team`-এর permission কেমন হবে (সংখ্যা ও অক্ষরে), intern কী দেখতে পারবে না — তা যুক্তিসহ লিখুন। তারপর VM-এ পরিকল্পনাটি বাস্তবায়ন করে `ls -ld` ও `id` output দিয়ে যাচাই করুন।

Intermediate — অ্যাসাইনমেন্ট ৫–৭

Assignment 05 — Package ও Service রুটিন

VM-এ `htop` ও `curl` ইনস্টল আছে কি না দেখুন; না থাকলে ইনস্টল করুন। তারপর: (ক) `apt update` ও `apt list --upgradable` চালিয়ে কয়টি update বাকি, তা লিখুন, (খ) `nginx` ইনস্টল করে চালু করুন ও `curl http://localhost` দিয়ে যাচাই করুন, (গ) nginx-কে boot-এ চালু হওয়ার জন্য enable করুন ও `systemctl is-enabled nginx` দিয়ে প্রমাণ করুন, (ঘ) একটি package-এর version ও repository `apt-cache policy` দিয়ে দেখান। প্রতিটি ধাপের output সংরক্ষণ করুন।

Assignment 06 — নিজের Service লেখা

Lab 08-এর ধাঁচে, কিন্তু ভিন্ন অ্যাপ দিয়ে একটি systemd service তৈরি করুন: একটি Bash স্ক্রিপ্ট প্রতি ৩০ সেকেন্ডে তারিখ ও `uptime` একটি ফাইলে লিখবে (`/var/log/myapp.log` নয় — প্রথমে `/tmp/myapp.log` ব্যবহার করুন, কেন তা ব্যাখ্যা করুন)। Service-টি `nobody` user-এ চালান, boot-এ enable করুন, `journalctl` দিয়ে log দেখান, service থামিয়ে আবার চালু করে আচরণ লিখুন।

Assignment 07 — Disk ও Resource রিপোর্ট

নিজের VM-এর একটি এক পৃষ্ঠার resource রিপোর্ট তৈরি করুন: load average, `free -h` (available ব্যাখ্যাসহ), সবচেয়ে বড় ৫টি ডিরেক্টরি (`du` দিয়ে), সবচেয়ে বেশি CPU ও RAM ব্যবহার করা ৫টি প্রসেস, ডিস্ক ব্যবহার শতাংশ, inode ব্যবহার। শেষে এক অনুচ্ছেদে লিখুন — এই সার্ভারে এখনই কোনো সমস্যা আছে কি না, থাকলে কী করবেন।

Advanced Troubleshooting — অ্যাসাইনমেন্ট ৮–১০

Assignment 08 — "ওয়েবসাইট খুলছে না" — ধাপে ধাপে তদন্ত

VM-এ nginx চালু থাকা অবস্থায় ইচ্ছাকৃতভাবে একটি সমস্যা তৈরি করুন (যেমন ভুল configuration, বা service বন্ধ, বা firewall-এ port 80 বন্ধ — যেকোনো একটি)। তারপর ধরুন আপনি কিছুই জানেন না: `systemctl status`, `journalctl`, `ss -tulpn`, `curl`, `ufw status` — এই ক্রমে তদন্ত করে মূল কারণ লিখুন, মেরামত করুন, আবার `curl` দিয়ে প্রমাণ করুন। তদন্তের প্রতিটি ধাপে কী দেখলেন ও কী সিদ্ধান্ত নিলেন, তা লিখুন।

Assignment 09 — "ডিস্ক ভরে গেছে" — নিরাপদ উদ্ধার

Lab 14-এর ধাঁচে একটি বড় ফাইল বা অসংখ্য ছোট ফাইল দিয়ে ডিস্ক চাপ তৈরি করুন। তারপর: (ক) `df -h` ও `df -i` — কোনটি শেষ হয়েছে, তা নির্ণয় করুন, (খ) `du` ও `find` দিয়ে দায়ী জায়গা খুঁজুন, (গ) কী মুছবেন ও কী মুছবেন না — তার তালিকা যুক্তিসহ লিখুন, (ঘ) নিরাপদে জায়গা খালি করে `touch` দিয়ে লেখা যায় কি না প্রমাণ করুন। ভুল cleanup-এর ঝুঁকি কী, তা এক অনুচ্ছেদে লিখুন।

Assignment 10 — "SSH-এ ঢোকা যাচ্ছে না" — Access সমস্যার তদন্ত

VM-এ SSH access-এ একটি সমস্যা তৈরি করুন (যেমন `authorized_keys` permission বদলে দেওয়া, বা UFW-তে SSH বন্ধ করা, বা ভুল user দিয়ে চেষ্টা — যেকোনো একটি, VM console access হাতে রেখে)। তারপর console থেকে তদন্ত করুন: service চলছে কি না, port listening আছে কি না, firewall নিয়ম, `~/.ssh` permission, `/var/log/auth.log`-এ কী লেখা আছে। মূল কারণ, মেরামতের ধাপ ও ভবিষ্যতে এড়ানোর উপায় লিখুন।

> **সতর্কতা:** Assignment ৮–১০-এ ইচ্ছাকৃত সমস্যা তৈরির আগে snapshot নিন। আসল সার্ভারে বা অন্য কারও সিস্টেমে এসব পরীক্ষা করবেন না।

পরিশিষ্ট C — Major Projects (তিনটি বড় প্রজেক্ট)

তিনটি প্রজেক্টই VM-এ করবেন। প্রতিটি প্রজেক্টের deliverable একটি ফোল্ডার বা রিপোর্ট — শুধু "করেছি" বললে হবে না, প্রমাণ (কমান্ড output, configuration ফাইল, স্ক্রিনশটের বিবরণ) জমা দিতে হবে।

Project A — Linux Workstation: নিজের Linux পরিবেশ তৈরি

> **Business Scenario:** আপনি একজন junior software engineer। আপনার দৈনন্দিন কাজ — কোড লেখা, Git, terminal, ডকুমেন্টেশন — সব Linux-এ করতে চান। একটি নির্ভরযোগ্য personal workstation দরকার, যা আপনি নিজে ইনস্টল, সাজিয়েছেন ও রক্ষণাবেক্ষণ করেন।

Requirements: Ubuntu Server 24.04 LTS বা Ubuntu Desktop 24.04 LTS VM; নিজের user account (sudo সহ); প্রয়োজনীয় tool ইনস্টল করা (Git, curl, Python 3, একটি text editor); home ডিরেক্টরির সুসংগঠিত কাঠামো; নিয়মিত update রুটিন; backup পরিকল্পনা।

Architecture — Project A

Host Laptop (Windows/macOS) → VirtualBox → Ubuntu VM (2 GB RAM, 20 GB Disk) → User: nahid (sudo) → Home: ~/code, ~/docs, ~/backup → Tools: Git, Python 3, curl, htop → Backup: ~/backup (rsync, সাপ্তাহিক)

Tasks ও Milestones:

1. Milestone 1 — VM ইনস্টল ও প্রথম configuration (Lab 01 অনুযায়ী)।
2. Milestone 2 — প্রয়োজনীয় software ইনস্টল ও সংস্করণ নথিভুক্ত করা।
3. Milestone 3 — Filesystem কাঠামো: `~/code`, `~/docs`, `~/downloads`, `~/backup` তৈরি ও উদ্দেশ্য লেখা।
4. Milestone 4 — Shell পরিবেশ: `.bashrc`-এ দরকারী alias (যেমন `ll`) যোগ করা, প্রতিটির ব্যাখ্যা লেখা।
5. Milestone 5 — Backup: `~/docs`-এর rsync backup ও restore পরীক্ষা।
6. Milestone 6 — Health-check রিপোর্ট (Lab 15-এর স্ক্রিপ্ট ব্যবহার)।

Deliverables: (১) `workstation-report.md` — system তথ্য, ইনস্টল করা software ও সংস্করণ, filesystem কাঠামো, alias তালিকা; (২) backup ও restore প্রমাণ; (৩) health-check রিপোর্ট।

Acceptance Criteria: VM reboot-এর পর সব service ও configuration অক্ষত থাকে; `git --version`, `python3 --version` চলে; backup থেকে একটি ফাইল restore করে দেখানো যায়; রিপোর্টে প্রতিটি সিদ্ধান্তের কারণ লেখা আছে।

Verification Checklist:

- Ubuntu 24.04 LTS প্রমাণ (`lsb_release -a`) আছে কি?
- সব software-এর সংস্করণ লেখা আছে কি?
- Home কাঠামো ও alias ব্যাখ্যাসহ আছে কি?
- Backup ও restore পরীক্ষা হয়েছে কি?

- Health-check রিপোর্টে OK/Watch/Action লেখা আছে কি?

Evaluation Rubric:

মানদণ্ড	Excellent	Good	Needs Work
System documentation	সব তথ্য, সংস্করণ ও কারণসহ সম্পূর্ণ	তথ্য আছে, কিছু কারণ অনুপস্থিত	তথ্য অসম্পূর্ণ বা ভুল
Filesystem ও shell	সুসংগঠিত, alias ব্যাখ্যাসহ	কাঠামো আছে, ব্যাখ্যা আংশিক	এলোমেলো, ব্যাখ্যা নেই
Backup ও restore	Restore পরীক্ষাসহ প্রমাণিত	Backup আছে, restore পরীক্ষা হয়নি	Backup নেই বা ভুল
Health check	সিদ্ধান্তসহ সম্পূর্ণ রিপোর্ট	রিপোর্ট আছে, সিদ্ধান্ত আংশিক	রিপোর্ট নেই

Optional Advanced Challenges: SSH key দিয়ে password-ছাড়া login; একটি cron job দিয়ে সাপ্তাহিক backup স্বয়ংক্রিয় করা (প্রথমে হাতে পরীক্ষা করে); `ufw` চালু করে শুধু প্রয়োজনীয় port খোলা।

Project B — Linux Server Administration: একটি সার্ভারের পূর্ণ দায়িত্ব

> **Business Scenario:** TechNova-র একটি Ubuntu সার্ভারে ৩ জন developer ও ১ জন intern কাজ করবেন। সার্ভারে একটি ওয়েব অ্যাপ (nginx), নিয়মিত backup, log পর্যবেক্ষণ ও নিরাপত্তা — সব আপনার দায়িত্ব।

Requirements: Ubuntu Server 24.04 LTS VM; ৪ জন user ও ২টি group; nginx service; UFW firewall; `/srv/technova` শেয়ার্ড ডিরেক্টরি; সাপ্তাহিক backup; health-check রুটিন।

Architecture — Project B

Internet → UFW (22, 80 only) → Ubuntu Server 24.04 → nginx (80) → /srv/technova (developers group, 2770) → Users: dev1, dev2, designer, intern1 → Logs: journalctl + /var/log/nginx → Backup: /backup (tar + rsync, সাপ্তাহিক) → Admin: health-check.sh

Tasks ও Milestones:

1. Milestone 1 — User/group তৈরি, শেয়ার্ড ডিরেক্টরি ও permission (Lab 04–০৫)।
2. Milestone 2 — nginx ইনস্টল, enable, `curl` দিয়ে যাচাই (Lab 07)।
3. Milestone 3 — Custom service (Lab 08) — অ্যাপ log `journalctl`-এ দেখা যায়।
4. Milestone 4 — UFW ও SSH hardening (Lab 12) — শুধু 22 ও 80 খোলা।
5. Milestone 5 — Backup ও restore পরীক্ষা (Lab 13)।
6. Milestone 6 — Health-check রিপোর্ট ও maintenance checklist (Lab 15)।

Deliverables: (১) `server-admin-report.md` — user/group তালিকা, permission ব্যাখ্যা, service তালিকা, firewall নিয়ম, backup পরিকল্পনা; (২) health-check রিপোর্ট; (৩) restore পরীক্ষার প্রমাণ।

Acceptance Criteria: প্রতিটি user শুধু তার অধিকার অনুযায়ী access পায় (intern শেয়ার্ড ফোল্ডারে লিখতে পারে, অন্যের private ফোল্ডারে নয়); nginx reboot-এর পরও চলে; UFW-তে শুধু 22 ও 80 ALLOW; backup থেকে একটি ফাইল restore করা যায়; কোনো failed service নেই।

Verification Checklist:

- `id` ও `ls -ld /srv/technova` পরিকল্পনা অনুযায়ী কি?
- `systemctl status nginx` active ও enabled কি?
- `ufw status`-তে শুধু প্রয়োজনীয় নিয়ম কি?
- `journalctl -u nginx -n 20`-তে নতুন error আছে কি?
- Backup checksum OK ও restore পরীক্ষা হয়েছে কি?

Evaluation Rubric:

মানদণ্ড	Excellent	Good	Needs Work
Users ও permissions	Least privilege, যাচাইসহ সম্পূর্ণ	কাঠামো ঠিক, যাচাই আংশিক	777 বা অতিরিক্ত অধিকার আছে
Services	সব service active, enabled, log পরিষ্কার	চলছে, কিন্তু enable/log আংশিক	Service ব্যর্থ বা বন্ধ
Security	UFW + SSH key, নিয়ম নথিভুক্ত	Firewall আছে, SSH আংশিক	Firewall বন্ধ বা সব port খোলা
Backup ও monitoring	Restore পরীক্ষাসহ, health রিপোর্ট সিদ্ধান্তসহ	Backup আছে, restore/রিপোর্ট আংশিক	Backup বা রিপোর্ট নেই

Optional Advanced Challenges: `logrotate` configuration বুঝে একটি log-এর rotation পরীক্ষা; fail2ban-এর ধারণা পড়ে SSH brute-force সুরক্ষা নিয়ে এক পৃষ্ঠা লেখা (ইনস্টল বাধ্যতামূলক নয়); disk quota-র ধারণা পড়ে `/srv/technova`-র ব্যবহার সীমা নিয়ে পরিকল্পনা লেখা।

Project C — Production Server Challenge: TechNova Production Server

> **Business Scenario:** TechNova একটি ছোট সফটওয়্যার কোম্পানি। আপনি তার junior Linux systems administrator। কোম্পানির Ubuntu সার্ভারে কোম্পানির ওয়েবসাইট (nginx) ও একটি internal অ্যাপ চলে। সোমবার সকালে একসাথে কয়েকটি সমস্যা ধরা পড়েছে — service থেমে গেছে, ডিস্ক প্রায় ভরা, CPU বেশি, একজন কর্মী SSH-এ ঢুকতে পারছেন না। আপনাকে সার্ভার প্রস্তুত, সুরক্ষিত, পর্যবেক্ষণ ও রক্ষণাবেক্ষণ করতে হবে, সমস্যাগুলোর তদন্ত করে backup থেকে recovery দেখাতে হবে।

Requirements: Ubuntu Server 24.04 LTS VM (Project B-র সার্ভার ব্যবহার করা যাবে); nginx + একটি custom service; UFW; backup (`/backup`); health-check স্ক্রিপ্ট; incident লগ (প্রতিটি সমস্যা, কারণ, মেরামত, সময়)।

Architecture — Project C

Users / Internet → UFW (22, 80) → Ubuntu Server 24.04 → nginx (website, port 80) + hello-app (systemd service) → /srv/technova (team data) → /var/log + journalctl (monitoring) → /backup (tar + rsync, off-disk কপির পরিকল্পনাসহ) → Admin (health-check.sh, incident log)

Tasks ও Milestones:

1. Milestone 1 — Baseline: সার্ভার প্রস্তুত করুন (user, service, firewall, backup) ও baseline health রিপোর্ট লিখুন।
2. Milestone 2 — Simulated Incident 1: nginx service failure — log পড়ে কারণ খুঁজে মেরামত করুন, incident লগে লিখুন।
3. Milestone 3 — Simulated Incident 2: Disk pressure — `df` / `du` দিয়ে কারণ খুঁজে নিরাপদ cleanup করুন।
4. Milestone 4 — Simulated Incident 3: Resource pressure — `top` / `vmstat` দিয়ে CPU/RAM bottleneck শনাক্ত করে দায়ী প্রসেস সামলান।
5. Milestone 5 — Simulated Incident 4: Access issue — একজন user SSH-এ ঢুকতে পারছেন না; permission, firewall ও `auth.log` দেখে মেরামত করুন।
6. Milestone 6 — Backup ও Recovery: একটি ফাইল মুছে backup থেকে restore করে প্রমাণ করুন; recovery সময় লিখুন।
7. Milestone 7 — Final Report: পুরো ঘটনা, প্রতিটি incident-এর root cause, মেরামত, ভবিষ্যতে এড়ানোর উপায় ও maintenance checklist।

Deliverables: (১) Baseline health রিপোর্ট; (২) Incident log (৪টি incident, প্রতিটিতে লক্ষণ, তদন্তের ধাপ, root cause, মেরামত, যাচাই); (৩) Backup/restore প্রমাণ; (৪) Final report — maintenance checklist সহ।

Acceptance Criteria: চারটি incident-এর প্রতিটির root cause log-এর প্রমাণসহ লেখা; প্রতিটি মেরামতের পর service `curl` /login দিয়ে যাচাই করা; backup থেকে restore সফল; final report-এ ভবিষ্যৎ প্রতিরোধের ব্যবস্থা (যেমন log rotation, monitoring, backup যাচাই) লেখা; কোনো ধ্বংসাত্মক কমান্ডের প্রমাণ নেই।

Verification Checklist:

- Baseline ও final health রিপোর্ট তুলনা করা যায় কি?
- প্রতিটি incident-এ `journalctl` /log-এর উদ্ধৃতি আছে কি?
- nginx ও custom service শেষে active কি?
- ডিস্ক ব্যবহার নিরাপদ সীমায় (80%-এর নিচে) কি?
- SSH login পরীক্ষা সফল কি?
- Restore করা ফাইলের checksum/বিষয়বস্তু মিলেছে কি?

Evaluation Rubric:

মানদণ্ড	Excellent	Good	Needs Work
		কারণ পাওয়া গেছে, প্রমাণ আংশিক	

Troubleshooting পদ্ধতি	ধাপে ধাপে, log-প্রমাণসহ, অনুমান নয়		অনুমান বা বারবার restart
Root cause	৪টি incident-এর সঠিক মূল কারণ	৩টি সঠিক	২ বা তার কম
Backup ও recovery	Restore পরীক্ষাসহ, সময় লেখা	Restore হয়েছে, সময়/প্রমাণ আংশিক	Restore হয়নি
Final report	সম্পূর্ণ, প্রতিরোধের ব্যবস্থাসহ	সম্পূর্ণ, প্রতিরোধ আংশিক	অসম্পূর্ণ

Optional Advanced Challenges: প্রতিটি incident-এর জন্য একটি করে "যদি এটি production-এ রাত ৩টায় হতো" — প্রথম ১৫ মিনিটের action plan লেখা; health-check স্ক্রিপ্টে disk 80% ছাড়ালে সতর্কবার্তা যোগ করা; backup-এর একটি কপি VM-এর বাইরে (যেমন হোস্টে) রাখার পরিকল্পনা লেখা।